Universidade Cruzeiro do Sul
Ciência da Computação

Aula 14 - Paradigma Orientado a Objetos – Parte 2

Disciplina: Paradigmas de Linguagens de Programação
Professor: Álvaro Colombero

Objetivos



- Compreender o conceito de herança. Definir polimorfismo e suas aplicações.
- Implementar classes em C++ utilizando os conceitos de herança e polimorfismo.

Paradigma Orientado a Objetos

- O POO surgiu a partir da necessidade de aumentar a produtividade dos programadores:
  - Reusabilidade.
  - Manutenibilidade.
  - Extensibilidade.

Paradigma Orientado a Objetos

- Herança:
  - Possibilita a reutilização de código.
  - Uma classe pode ser declarada como subclasse de outra que é conhecida como superclasse ou classe mãe.
  - A subclasse herda as variáveis e métodos de sua classe mãe.
  - Normalmente representa uma associação do tipo é-um ou é-uma..

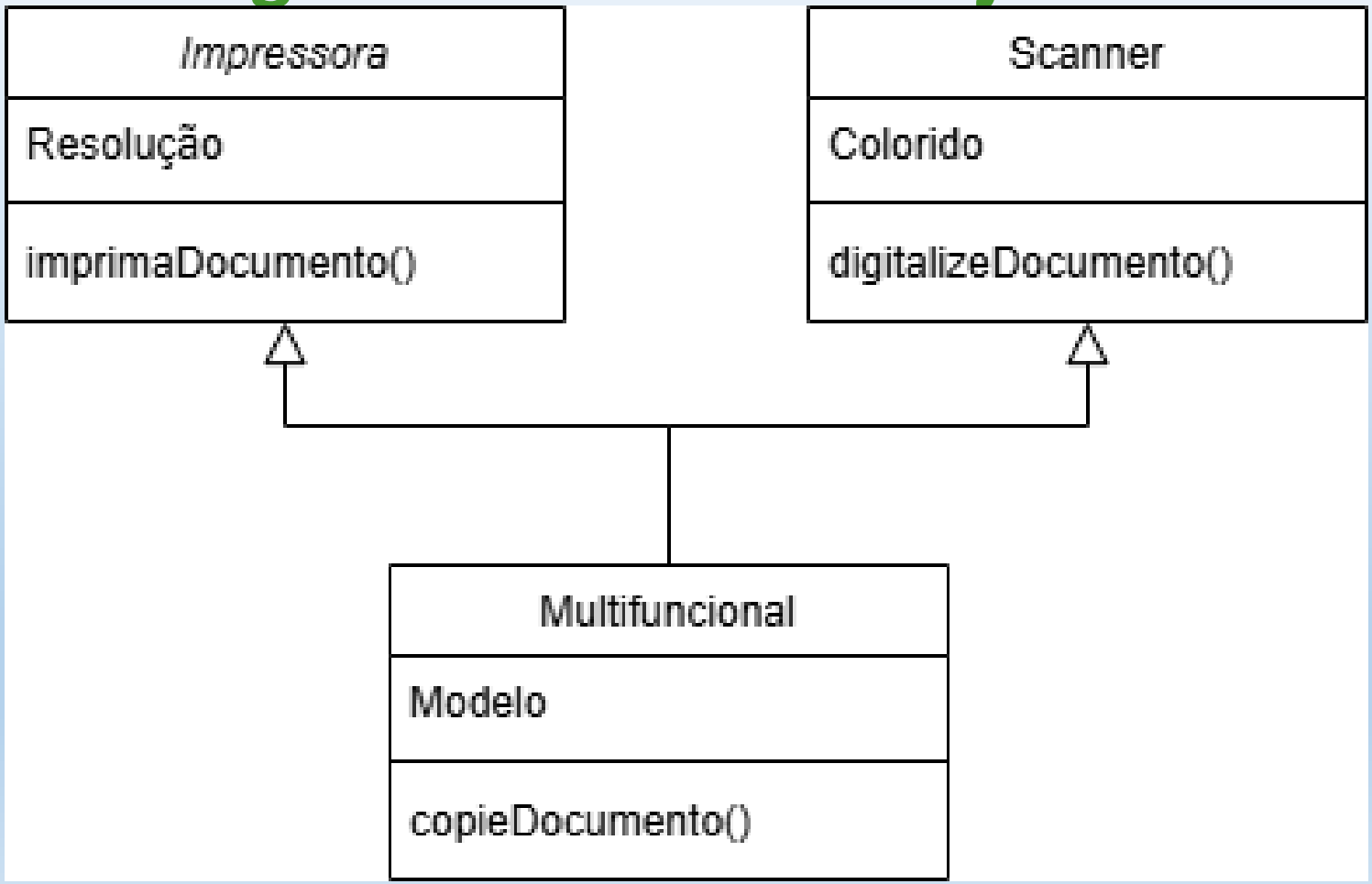
Paradigma Orientado a Objetos

- A subclasse difere da superclasse da seguinte maneira:
  - A superclasse pode definir membros privados (a subclasse não tem acesso a eles).
  - A subclasse pode adicionar membros aos herdados da superclasse.
  - A subclasse pode modificar o comportamento de um ou mais métodos herdados.

Paradigma Orientado a Objetos

- Herança simples:
  - A classe pode ter apenas uma superclasse (Java e C#).
- Herança múltipla:
  - A classe pode ter mais de uma superclasse (C++ e Python).
- Vantagens da herança múltipla:
  - Aumenta a reutilização de código.
- Desvantagens da herança múltipla:
  - Complexidade e eficiência.

Paradigma Orientado a Objetos



Paradigma Orientado a Objetos

Exemplo 1: Analise o programa em C++ a seguir que implementa as classes do diagrama apresentado.

Que problemas podem ocorrer com o uso da herança múltipla.

Paradigma Orientado a Objetos

```cpp
#include <iostream>
using namespace std;
//Classe base 1
class Impressora {
public:
    void imprimaDocumento() {
        cout << "Estou imprimindo um documento." << endl;
    }
};
// Classe base 2
class Scanner {
public:
    void digitalizeDocumento() {
        cout << "Estou digitalizando um documento." << endl;
    }
};
```

```cpp
// Classe com herança múltipla
class Multifuncional : public Impressora, public Scanner {
public:
    void copieDocumento() {
        cout << "Estou copiando um documento" << endl;
        cout << "******" << endl;
        digitalizeDocumento();
        imprimaDocumento();
        cout << "******" << endl;

    }
};
int main() {
    Multifuncional hp3000;
    hp3000.imprimaDocumento();
    hp3000.digitalizeDocumento();
    hp3000.copieDocumento();

    return 0;
}
```

Atividade 1



- Em um programa em Java que apresente o mesmo comportamento do exemplo apresentado em C++, que alterações na estrutura do programa devem ser feitas?

Atividade 1



- Em um programa em Java que apresente o mesmo comportamento do exemplo apresentado em C++, que alterações na estrutura do programa devem ser feitas?

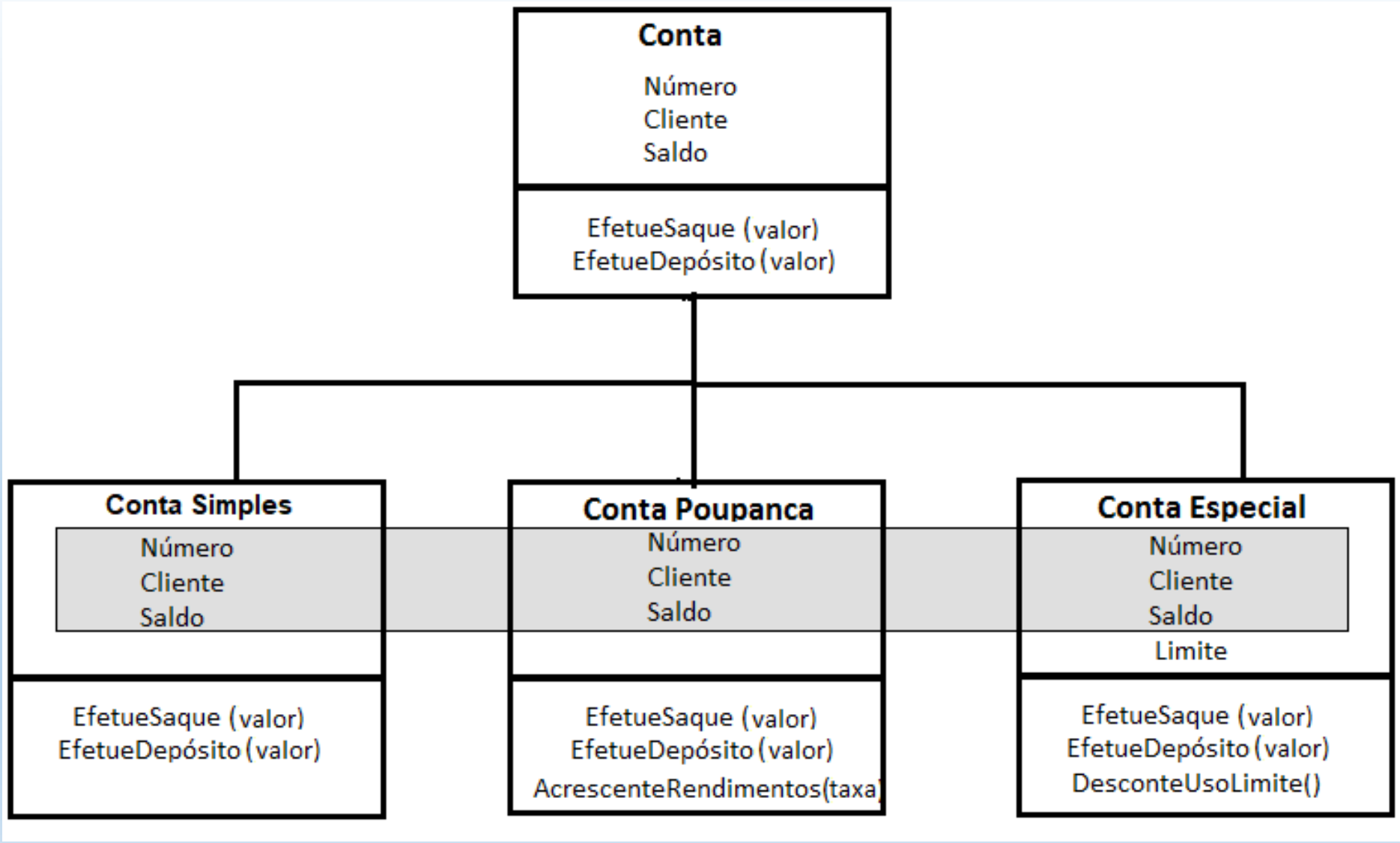
```
public interface Impressora {
        void imprimaDocumento();
}
public interface Scanner {
        void digitalizeDocumento();
}
public class Multifuncional implements Impressora, Scanner {
        // Implementação dos métodos das duas interfaces
}
```

Paradigma Orientado a Objetos

- **Polimorfismo** é a ligação tardia de uma chamada a uma ou várias implementações diferentes de um método em uma hierarquia de herança.
  - Permite a declaração de um objeto de uma classe base para referenciar instâncias de diferentes classes derivadas;
  - Ao invocar a execução de um método, automaticamente será chamado o da classe derivada correspondente.
  - Polimorfismo estático (sobrecarga) X Polimorfismo dinâmico (sobrescrita).
  - Bastante utilizado em padrões de projeto de software tais como: Abstract Factory, Composite, Observer e Strategy.

Paradigma Orientado a Objetos

- **Classe abstrata** é uma classe que contém um ou mais métodos abstratos.
  - Não se pode criar instâncias de classes abstratas.
  - As classes derivadas das classes abstratas devem implementar seus métodos abstratos.
  - Interfaces em Java são uma extensão deste conceito, porém elas não podem incluir variáveis, construtores ou métodos não abstratos.


Conta
Número
Cliente
Saldo
EfetueSaque (valor)
EfetueDepósito (valor)
Conta Simples
Número
Cliente
Saldo
EfetueSaque (valor)
EfetueDepósito (valor)
Conta Poupanca
Número
Cliente
Saldo
EfetueSaque (valor)
EfetueDepósito (valor)
AcrescenteRendimentos(taxa)
Conta Especial
Número
Cliente
Saldo
Limite
EfetueSaque (valor)
EfetueDepósito (valor)
DesconteUsoLimite()

Paradigma Orientado a Objetos

- Histórico:
  - Crescente demanda por soluções de software e aumento na complexidade das mesmas (Crise de Software);
  - Necessidade de aumentar a produtividade dos programadores:
    - Reusabilidade.
    - Manutenibilidade.
    - Extensibilidade.

Paradigma Orientado a Objetos

- **Reusabilidade** é a utilização de conceitos ou soluções previamente elaboradas para a criação de um novo software.
- **Extensibilidade** refere-se às condições associadas à facilidade de se estender o software:
- – Adição de novas funcionalidades.
  - **Manutenibilidade** refere-se às condições associadas à facilidade de se alterar o software:
    - Correções de bugs.
    - Alteração de funcionalidades.

Paradigma Orientado a Objetos

- As classes são estruturas fracamente acopladas que facilitam alterações bem como o reaproveitamento de códigos prontos.
- Quanto mais específicas forem as implementações de cada classe (quanto menor seu número de responsabilidades), melhor será sua **Reusabilidade, Extensibilidade** e **Manutenibilidade**).
- A **Herança** é a forma de reusabilidade mais marcante na POO:
  - Com ela podemos criar novas classes adicionando menos código.
- Outra importante ferramenta é o polimorfismo, que permite chamar métodos de diferentes classes derivadas, a partir de um único comando da classe base.